

# Exercise 0: Waiting for Godot

## Overview

In this exercise, you will build a project that incorporates the basic elements of a game loop to read input from the user and update a player character's representation in Godot. This will include creating a scene and nodes (game objects), editing source code, and connecting source functions to game objects to accomplish the goal.

## Structure

The exercise is broken into these main parts:

- 1) Build a scene for the player character, then one for remaining scene elements using resources provided.
- 2) Capture input from the player and invoke a game script when input is received.
- 3) Write the game script to implement the movement behavior.
- 4) Run and test the implementation before submission.

The project should use a view port size of 640x480 pixels.

## Scenes

The exercise contains two scenes – a Player scene for the player character elements and a Level scene that contains that background and player. The Level scene should contain the movement logic.

### Player Scene

The **Player** scene (*Player.tscn*) should have a **Node2D** as its root (named *PlayerNode*) containing a single **Sprite2D** (*PlayerSprite*). The sprite should use the default *icon.svg* image and be offset so that its left edge is within the viewport, as follows:

|                             |                       |
|-----------------------------|-----------------------|
| <b>Texture</b>              | <i>icon.svg</i>       |
| <b>Transform → Position</b> | <i>x = 64, y = 64</i> |

All other attributes should be unmodified; i.e., they should hold the default values.

### Level Scene: Nodes

The **Level** scene (*Level.tscn*) should have a **Node2D** as its root (named *LevelNode*) which contains a link to the **Player** scene named (named *Player*) and a **Sprite2D** for the background image (*Background*). The background sprite should use the *background.png* image and be scaled and offset so that its left, right, and top edges are within the viewport, as follows:

|                             |                         |
|-----------------------------|-------------------------|
| <b>Texture</b>              | <i>background.png</i>   |
| <b>Transform → Position</b> | <i>x = 320, y = 320</i> |
| <b>Transform → Scale</b>    | <i>x = 2.5, y = 2.5</i> |

All other attributes should be unmodified; i.e., they should hold the default values.

## Level Scene: Scripting

In the **Level** scene, the player script file (**Player.cs**) should be attached to the **Player** object. The script file must be written in the C# language and should process user input to change the velocity of the **Player** object:

Max Velocity:                      **256 pixels / second** (*magnitude*)  
Acceleration:                    **512 pixels / second<sup>2</sup>** (*magnitude*)

When any keys are pressed and there is a net value in the vertical and/or horizontal, the acceleration (magnitude of vector) should be applied as follows:

- **ui\_left:**        negative 'x'
- **ui\_right:**      positive 'x'
- **ui\_up:**         negative 'y'
- **ui\_down:**       positive 'y'

For example, if only one directional key is pressed, the acceleration is applied fully to only that direction; on the other hand, if two keys are pressed that do not cancel out (e.g., right and up), the acceleration vector's magnitude must max the acceleration, so the x and y components must each be less. The values can be derived using vector algebra and / or geometry. Finally, if two keys on the same axis (opposite direction) are pressed, they will cancel each other out (e.g., up and down).

To update the object correctly, follow these steps:

- 1) Identify the direction of acceleration (if any).
- 2) Calculate the acceleration vector (from the direction); it is possible for this to be a zero vector.
- 3) Update the velocity vector based on the acceleration – but make sure to observe the maximum speed!
- 4) Update the position based on the velocity.

**NOTE:** Make sure to account for the amount of time that has passed since the object was last updated; otherwise, your calculations will be inconsistent!

## Submissions

Students will submit a **zip file** named **Ex0.zip** containing the following files at the end of this exercise on Canvas:

- Level.tscn
- Player.tscn
- Player.cs

Place them in the **root directory** of your zip file, not in a subdirectory. Do not submit any other files.